

OE-EC604C

Object Oriented Programming

Combined Module Notes

Modules 1 – 9

Semester VI • 3L:0T:0P • 3 Credits

CGEC | B.Tech ECE (2023–27)

Biraj Sarkar

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Contents

Module 1: Evolution of Programming Paradigm	5
1 Evolution of Programming Paradigm	5
1.1 Paradigms Evolution Timeline	5
1.2 Structured vs. Object-Oriented Development	5
2 Core Object-Oriented Programming Concepts	6
2.1 Objects and Classes	6
2.2 Data Encapsulation and Data Abstraction	6
2.3 Inheritance	6
2.4 Polymorphism	7
2.5 Dynamic Binding and Message Passing	7
Module 2: C++ Streams, Namespaces & Operators	11
1 C++ Streams, Namespaces & Operators	11
1.1 Stream-Based Input/Output	11
1.2 Namespaces	11
1.3 Scope Resolution Operator (::)	11
1.4 Variable Declaration and References	12
2 Parameter Passing & Functions	12
2.1 Parameter Passing Mechanisms	12
2.2 Inline Functions	13
2.3 Function Overloading	14
2.4 Default Arguments	14
Module 3: Classes, Objects and Member Definitions	18
1 Classes, Objects and Member Definitions	18
1.1 Specifying a Class and Access Specifiers	18
1.2 Defining Class Member Functions	18
2 Static Data Members and Member Functions	19
2.1 Static Data Members	19
2.2 Static Member Functions	19
3 Friend Functions and Friend Classes	20
3.1 Friend Functions	20
3.2 Friend Classes	22
4 Object Memory and Dynamic Allocation	22
4.1 Memory Considerations for Objects	23
4.2 Dynamic Memory Management: <code>new</code> and <code>delete</code>	23

Module 4: Fundamentals of Constructors	28
1 Fundamentals of Constructors	28
1.1 Rules and Characteristics	28
1.2 Types of Constructors	28
2 Deep Copying and Shallow Copying	28
2.1 Shallow Copy vs. Deep Copy	29
3 Constructor Overloading & Destructors	30
3.1 Constructor Overloading and Default Arguments	30
3.2 Destructors	30
Module 5: Fundamentals of Operator Overloading	35
1 Fundamentals of Operator Overloading	35
1.1 General Rules and Syntax	35
2 Binary & Unary Operator Overloading	36
2.1 Unary Operator Overloading	36
2.2 Binary Operator Overloading	36
2.3 Stream Insertion (<<) and Extraction (>>) Overloading	37
3 Type Conversions	37
3.1 Basic Type to Class Type	38
3.2 Class Type to Basic Type	38
Module 6: Fundamentals of Inheritance & Derivation Levels	43
1 Fundamentals of Inheritance & Derivation Levels	43
1.1 Base and Derived Classes	43
1.2 Types of Inheritance	43
2 Constructor & Destructor Execution Sequence	44
2.1 Invocation Sequence Rules	44
3 The Diamond Inheritance Ambiguity	45
3.1 The Diamond Problem and Virtual Base Classes	45
4 Containership vs. Inheritance	46
Module 7: Fundamentals of Polymorphism	50
1 Fundamentals of Polymorphism	50
1.1 Pointers to Objects and Polymorphic Assignments	50
1.2 The <code>this</code> Pointer	50
2 Compile-Time vs. Run-Time Polymorphism	50
2.1 Virtual Functions and Late Binding	50
3 Pure Virtual Functions and Abstract Classes	52
3.1 Pure Virtual Functions	53
3.2 Abstract Classes	53

4 Virtual Destructors & Memory Leaks	53
4.1 Destructor Call Failures	54
Module 8: C++ File Streams Hierarchy	58
1 C++ File Streams Hierarchy	58
1.1 The Stream Class Hierarchy	58
2 Opening & Closing Files	58
2.1 File Open Methods	58
2.2 File Modes	59
3 File Pointers and Seek Operations	59
3.1 Seek and Tell Functions	59
4 Binary & Random Access Operations	60
4.1 Object Writing using <code>write()</code> and <code>read()</code>	60
4.2 Random Access Database Record Update	60
Module 9: Exception Handling Mechanism	65
1 Exception Handling Mechanism	65
1.1 Basic Syntax: <code>try</code> , <code>throw</code> , and <code>catch</code>	65
1.2 Exception Propagation & Stack Unwinding	65
1.3 Multiple Catches, Catch-All, and Re-throwing	66
2 Generic Programming using Templates	66
2.1 Function Templates	66
2.2 Class Templates	67

Module 1

Evolution of Programming Paradigm

1. Evolution of Programming Paradigm

A programming paradigm defines the style, approach, and philosophy of designing and writing software code. As programs grew in size and complexity, paradigms evolved to improve code readability, reuse, and maintainability.

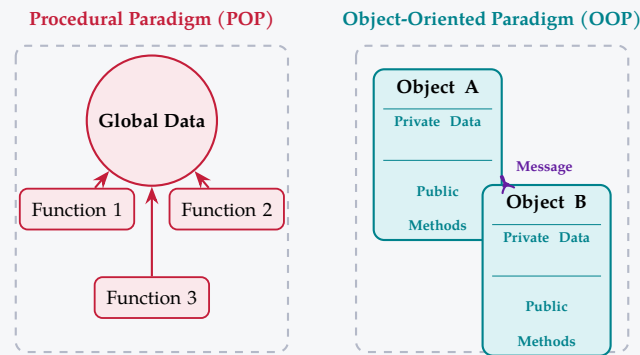
1.1 Paradigms Evolution Timeline

- 1. Unstructured Programming:** The earliest style where the program consists of a single continuous block of statements (often containing global variables) using jump statements like `goto` for flow control. It becomes difficult to maintain as code grows ("spaghetti code").
- 2. Procedural / Structured Programming:** Programs are organized into subroutines, modules, or functions. Flow control is structured using blocks like loops (`for`, `while`) and conditional statements (`if`, `switch`). Focus is on the **process or algorithm** rather than data.
- 3. Object-Oriented Programming (OOP):** A paradigm where programs are organized around **objects** (representing real-world entities) containing data (attributes) and methods (behaviors). Focus is on **data security and encapsulation** rather than procedures.

1.2 Structured vs. Object-Oriented Development

- **Structured/Procedure-Oriented Programming (POP):** Views a program as a sequence of procedural steps or function calls. Global data is vulnerable because it can be accessed and modified freely by any function.
- **Object-Oriented Programming (OOP):** Views a program as a collection of self-contained, interacting objects. Data is tied closely to the functions that operate on it, preventing accidental outside modification.

Procedural Paradigm (POP) vs. Object-Oriented Paradigm (OOP)



2. Core Object-Oriented Programming Concepts

OOP seeks to model software systems in terms of real-world entities. Below are the foundational concepts of OOP:

2.1 Objects and Classes

Class: A user-defined data type that acts as a blueprint, template, or prototype for creating objects. It defines the properties (data members) and actions (member functions) common to all objects of that class.

Object: A basic run-time entity of an object-oriented system that represents a real-world entity. It is an instance of a class, possessing its own state (data values) and behavior.

2.2 Data Encapsulation and Data Abstraction

Data Encapsulation: The mechanism of binding together code (member functions) and the data it manipulates (data members) into a single unit (class). It implements data hiding, keeping data safe from outside interference.

Data Abstraction: The act of representing essential features of a system without including the background details or implementation explanations. Users interact only with the interface, hiding complex execution.

2.3 Inheritance

Inheritance: The process by which objects of one class acquire the properties and behaviors of another class. It supports hierarchical classification and promotes **code reusability** by allowing new classes (derived classes) to extend existing ones (base classes).

Base Class (Parent Class): The class whose properties are inherited.

Derived Class (Child Class): The class that inherits from the base class.

2.4 Polymorphism

Polymorphism: The ability of an interface to take multiple forms (from Greek, meaning "many forms"). It allows a single function name, operator symbol, or interface to behave differently depending on the context.

There are two main classifications:

- **Compile-Time (Static) Polymorphism:** Resolved at compile time (e.g., function overloading, operator overloading).
- **Run-Time (Dynamic) Polymorphism:** Resolved at runtime during execution (e.g., virtual functions, method overriding).

2.5 Dynamic Binding and Message Passing

Dynamic Binding: The mechanism where the code associated with a given function call is not resolved until runtime. It is associated with inheritance and run-time polymorphism.

Message Passing: The communication process between objects. Objects interact by sending requests to execute methods of other objects, passing arguments and receiving results.

Quick Revision — Module I

Paradigm & OOP Concepts | OE-EC604C

Key Concepts at a Glance

Class Structure	<pre>class ClassName { private: // data public: // methods };</pre>
The 4 Pillars of OOP	Encapsulation (data hiding), Abstraction (interface separation), Inheritance (code reuse), Polymorphism (many forms).
Access Modifiers	private: Accessible only within class (default). public: Accessible from anywhere. protected: Accessible within class and child classes.

One-Line Definitions

- **Paradigm:** A structured method or style of programming defining program organization.
- **Class:** A user-defined logical blueprint defining data and member functions.
- **Object:** A physical instance of a class representing a real-world entity.
- **Encapsulation:** Wrapping data and code together into a single unit (class) to prevent global access.
- **Data Hiding:** Restricting direct access to internal data members using access specifiers.
- **Abstraction:** Hiding background implementation details and exposing only the interface.
- **Inheritance:** The mechanism of deriving new classes from existing classes to achieve code reuse.
- **Polymorphism:** The capability of a function, operator, or object to act differently in different contexts.
- **Dynamic Binding:** Resolving method calls at runtime rather than compile time.
- **Message Passing:** Communication model where objects invoke member functions of other objects.

Frequently Asked / Viva Questions

Q1. State the main limitation of Unstructured Programming.

Unstructured programming relies heavily on jump commands (like `goto`), making the control flow complex and highly difficult to trace, maintain, or debug.

Q2. What is the core difference in how POP and OOP view data?

Frequently Asked / Viva Questions (contd.)

Procedure-Oriented Programming (POP) views data as passive elements manipulated by active procedures. OOP views data as critical assets tied securely to active functions.

Q3. Define the difference between a Class and an Object.

A class is a logical template or blueprint (allocates no memory). An object is a physical instance of that template created in memory (allocates memory).

Q4. Why does OOP offer better data security than POP?

OOP hides data members inside classes using the `private` access specifier, preventing outside functions from modifying the variables. POP global data is accessible to all.

Q5. How does Encapsulation differ from Abstraction?

Encapsulation is the *wrapping* or *containment* of data (data hiding). Abstraction is the *hiding of complexity* by showing only the external interface.

Q6. What is the primary benefit of Inheritance?

Code reusability. It allows developer to extend base class properties without rewriting the code, reducing testing and development time.

Q7. Explain the term "Polymorphism" with a real-world example.

A single instruction performs different actions based on target. E.g., the "+" symbol performs addition for numbers, but performs concatenation for strings.

Q8. Define static binding vs. dynamic binding.

Static binding (early binding) resolves function calls at compile time. Dynamic binding (late binding) resolves function calls at runtime during execution.

Q9. What is the role of access specifiers in C++?

Access specifiers (`private`, `public`, `protected`) control the visibility and accessibility of class members from outside classes.

Q10. What is meant by the term "Message Passing"?

It is the process by which an object requests execution of a method belonging to another object, passing data parameters and receiving return values.

Q11. Can you have an object without a class in C++?

No. In C++, every object must belong to a predefined class which defines its structure and member specifications.

Q12. What is "Spaghetti Code" and in which paradigm does it commonly occur?

Spaghetti code is program code with highly convoluted control flow, usually caused by multiple jump statements (`goto`). It occurs in unstructured programming.

Comparison of POP and OOP Paradigms

Parameter	Procedure-Oriented (POP)	Object-Oriented (OOP)
Approach	Top-down design methodology.	Bottom-up design methodology.
Focus	Focuses on the algorithm/process.	Focuses on the data assets.
Data Access	Functions share global data (unsafe).	Objects hide data using private scope (secure).
Reusability	Limited to function calls.	Achieved via class inheritance.
Examples	C, Pascal, Fortran.	C++, Java, Python, C#.

Module 2

C++ Streams, Namespaces & Operators

1. C++ Streams, Namespaces & Operators

C++ maintains backwards compatibility with C but introduces modern syntax and strict type checking to simplify development.

1.1 Stream-Based Input/Output

Unlike C which uses formatted functions like `printf` and `scanf`, C++ implements stream-based I/O. Streams are continuous sequences of bytes representing data flow.

- **cout:** Standard output stream (connected to console), used with the stream insertion operator (`<<`).
- **cin:** Standard input stream (connected to keyboard), used with the stream extraction operator (`>>`).
- **Type Safety:** Operator overloading allows streams to automatically detect variables' data types (no format specifiers like `%d` or `%f` required).

1.2 Namespaces

Namespace: A declarative region that provides a scope to the identifiers (names of types, functions, variables, etc.) inside it. Namespaces are used to organize code into logical groups and prevent name conflicts (clashing) in large projects.

The C++ Standard Library is defined within a namespace named `std`. To use identifiers like `cout` or `cin`, we must specify their namespace:

using scope resolution: `std::cout << "Hello";` (1)

using declaration: `using std::cout; cout << "Hello";` (2)

using directive: `using namespace std; cout << "Hello";` (3)

1.3 Scope Resolution Operator (::)

In C, if a local variable hides a global variable with the same name, there is no direct way to access the global variable. C++ introduces the scope resolution operator (`::`) to resolve scope conflicts.

Scope Resolution Syntax

Global Variable Access: `::variable_name` (4)

It is also used for:

- a. Accessing static members of a class: `ClassName::static_member`.
- b. Defining class member functions outside the class declaration.
- c. Resolving namespace paths: `NamespaceName::identifier`.

Code Example: Global variable access via Scope Resolution

```
#include <iostream>
int x = 10; // Global variable
int main() {
    int x = 20; // Local variable hiding global x
    std::cout << "Local x = " << x << std::endl;    // Prints 20
    std::cout << "Global x = " << ::x << std::endl;    // Prints 10
    return 0;
}
```

1.4 Variable Declaration and References

- **Declaration at Point of Use:** C requires variable declarations at the start of a block. C++ allows declaring variables anywhere in the code, immediately before their first use.
- **Reference Variables (Aliases):**

Reference Variable: An alias or alternative name for an already existing variable. It shares the same memory space as the original variable. Modifying the reference modifies the original variable.

Declaration Syntax:

`int x = 10; int &y = x; // y is a reference to x` (5)

Reference variables must be initialized during declaration and cannot be re-bound to another variable.

2. Parameter Passing & Functions

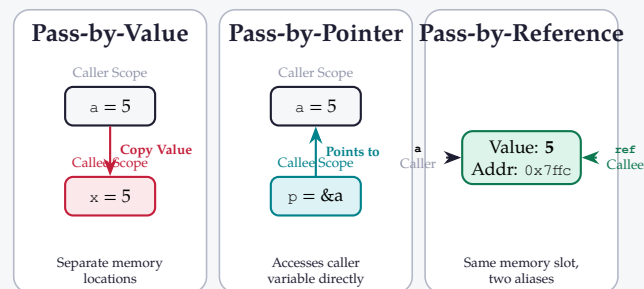
C++ extends function properties by introducing call-by-reference, inline compilation, and parameter defaults.

2.1 Parameter Passing Mechanisms

1. **Pass-by-Value:** A copy of the actual argument is passed to the function parameter. Changes inside the function do not affect the original variable.

2. **Pass-by-Pointer:** The address of the argument is passed. The function dereferences the pointer to access/modify the original variable.
3. **Pass-by-Reference:** The function parameter becomes an alias (reference) to the actual argument. No new memory is allocated, and the function directly manipulates the caller's variables.

Memory Behavior of Parameter Passing



Code Example: Swapping variables using references

```
#include <iostream>
void swap(int &x, int &y) { // x and y are references to caller variables
    int temp = x;
    x = y;
    y = temp;
}
int main() {
    int a = 5, b = 10;
    swap(a, b); // Swaps values of a and b directly
    std::cout << "a = " << a << ", b = " << b; // Prints a = 10, b = 5
    return 0;
}
```

2.2 Inline Functions

Function calls introduce overhead: pushing arguments onto stack, saving registers, jumping to function address, restoring state on return. For small functions, the overhead can exceed the execution time of the function body.

Inline Function: A function that is expanded in line at the point of call by the compiler instead of being invoked via standard function call stack jumps.

Syntax:

```
inline int add(int a, int b) { return a + b; } (6)
```

Key Rules and Compiler Discretion: The `inline` keyword is a **request**, not a command. The compiler can ignore it and compile the function normally if:

- The function contains loops (`for`, `while`), `switch` statements, or `goto`.
- The function is recursive.

- The function contains static variables.
- The function returns a value but does not contain a return statement.

2.3 Function Overloading

Function overloading is a compile-time polymorphism feature allowing multiple functions to share the same name with different parameters.

Function Overloading: The capability to declare multiple functions with the same name within the same scope, provided their signatures (parameter list) are unique.

Resolution Criteria: The compiler distinguishes overloaded functions based on:

1. **Number of parameters:** e.g., `void display(int)` vs `void display(int, int)`.
2. **Type of parameters:** e.g., `void display(int)` vs `void display(double)`.
3. **Order of parameters:** e.g., `void display(int, double)` vs `void display(double, int)`.

Note: Functions cannot be overloaded based solely on their return types.

2.4 Default Arguments

Default arguments allow functions to assign a default value to a parameter if the caller does not provide an argument for it. **Syntax:**

```
void printLine(char c = '*', int len = 40); // default values (7)
```

Critical Rule: Default arguments must be assigned from **right to left**. We cannot assign a default value to a parameter unless all parameters to its right also have default values.

- **Correct:** `void func(int a, int b = 10, int c = 20);`
- **Incorrect:** `void func(int a = 10, int b, int c = 20);` // b lacks default

Quick Revision — Module II

Moving from C to C++ | OE-EC604C

Key Formulas and Syntax

Reference Variable	<code>int x = 5; int &ref = x; // ref shares memory with x</code>
Inline Function	<code>inline int square(int x) { return x * x; }</code>
Scope Resolution	<code>::variable</code> (accesses global variable hiding in local scope)
Default Parameters	<code>void value(int p, int n = 5, float r = 0.1); // right-to-left</code>

One-Line Definitions

- **Reference Variable:** A constant pointer to variable that acts as an alternative name (alias) for that variable.
- **Namespace:** A scope region containing definitions to prevent name clashes in large programs.
- **Scope Resolution (::):** An operator used to access global variables or define class methods externally.
- **Inline Function:** A function that is compiled by replacing its calls directly with the function code.
- **Function Overloading:** Declaring multiple functions sharing the same name but unique parameters.
- **Default Argument:** A value assigned to a function parameter in its prototype used if argument is omitted.
- **Pass-by-Reference:** Passing parameters by sharing the caller's variable memory directly via references.
- **Type Safety:** Strict compile-time checks verifying variable types and function arguments matching.

Frequently Asked / Viva Questions

Q1. Why does C++ input/output streams compile with greater type safety than C standard I/O?

C++ streams use overloaded insertion/extraction operators to automatically detect variable types, removing format specifier errors.

Q2. Can we declare reference variables without initialization in C++?

No. A reference must be initialized to bind to a variable during declaration. E.g.,
`int &ref;` causes a compile error.

Frequently Asked / Viva Questions (contd.)**Q3. Can a reference variable be re-assigned to reference another variable later?**

No. Once a reference variable is initialized to a target variable, it cannot be changed to reference any other variable.

Q4. How does the scope resolution operator access global variables?

Adding the `::` prefix tells the compiler to search for the identifier in the global scope instead of the local stack block.

Q5. What is the primary benefit of using inline functions?

It eliminates function call overhead (saving registers, jumping, stack frames), improving execution speed for small functions.

Q6. Under what conditions will the C++ compiler ignore an inline request?

If the function is large, recursive, contains loops (`for`, `while`), `switch/goto` statements, or static variables.

Q7. Can we overload a function based only on its return type?

No. The compiler resolves overloading by comparing parameter lists (number, type, order). Return types are ignored.

Q8. State the mandatory rule for declaring default arguments in C++.

Default arguments must be added from right to left in the parameter list. No default argument can precede a non-default one.

Q9. What is the difference between pass-by-pointer and pass-by-reference?

Pass-by-pointer passes a memory address (can be NULL, requires dereferencing). Pass-by-reference passes an alias (cannot be NULL, behaves like local variable).

Q10. Explain the use of the `using namespace std;` statement.

It imports all identifiers from the standard library namespace (`std`) into global scope, removing the need to write `std::` prefixes.

Q11. Why are inline functions preferred over preprocessor macros (`#define`)?

Inline functions are compiled with full type-checking and follow scope and access rules, whereas macros are simple text replacements.

Q12. What is a name collision/clash and how do namespaces solve it?

A name collision occurs when two libraries define classes or functions with identical names. Namespaces solve this by wrapping definitions in unique scope scopes.

Comparison of Parameter Passing Methods

Parameter	Pass-by-Value	Pass-by-Pointer	Pass-by-Reference
Memory Allocation	Allocates new local storage for parameter copy.	Allocates storage for pointer address (4/8 bytes).	No new memory allocated. Shares caller's slot.
Null Values	Not applicable.	Pointer can be <code>nullptr</code> .	Reference cannot be null.
Syntax	Simple: <code>func(a)</code> .	Complex: <code>func(&a)</code> and <code>*ptr</code> .	Simple: <code>func(a)</code> with parameter <code>&ref</code> .
Modifies Caller?	No, operations are on copy.	Yes, via pointer dereferencing.	Yes, directly via alias.

Module 3

Classes, Objects and Member Definitions

1. Classes, Objects and Member Definitions

In C++, classes serve as user-defined data types that bind data and operations together, forming the foundation of encapsulation.

1.1 Specifying a Class and Access Specifiers

A class is specified using the `class` keyword. Inside a class, variables (data members) and functions (member functions) are declared under three access levels:

- **private:** Members declared private are accessible only inside member functions of the same class (default access level).
- **public:** Members are accessible from any part of the program where the class object is visible.
- **protected:** Members are accessible within the class itself and its derived (inherited) classes.

1.2 Defining Class Member Functions

Member functions can be defined in two ways:

1. **Inside the Class (Inline Definition):** Functions defined directly within the class declaration are implicitly treated as `inline` by the compiler.
2. **Outside the Class (Scope Resolution):** Functions are declared inside the class and defined outside using the scope resolution operator (`::`).

Member Function Syntax Outside Class

```
Return_Type  Class_Name::Function_Name(Arguments)  {  Body  }  (8)
```

Code Example: Inside vs. Outside class definitions

```
#include <iostream>
class Rectangle {
private:
    double length, width;
public:
    // Defined inside the class: treated as inline
    void setData(double l, double w) {
        length = l;
        width = w;
    }
    double getArea(); // Declared inside class, defined outside
};

// Defined outside using scope resolution
double Rectangle::getArea() {
    return length * width;
}
```

2. Static Data Members and Member Functions

Normally, every object has its own separate copy of all data members. However, static members allow sharing of data across all instances of a class.

2.1 Static Data Members

Static Data Member: A variable declared with the `static` keyword inside a class. Only one copy of the static member is created for the entire class and is shared by all objects of that class, regardless of how many objects are instantiated.

Key Characteristics:

- **Lifetime:** It is initialized to 0 when the first object is created (unless initialized explicitly). It persists throughout the program execution.
- **Storage:** Allocated in the global static memory segment, not within the individual objects.
- **Definition Rule:** It must be defined and initialized outside the class block using the scope resolution operator.

2.2 Static Member Functions

A static member function can only access other static data members or static functions of the same class. **Characteristics:**

- Called using the class name: `ClassName::functionName()`. No object instance is required.

- Does not possess a `this` pointer (since it is not bound to a specific object instance).

Code Example: Tracking Object Counts using Static Members

```
#include <iostream>
class Test {
private:
    int id;
    static int count; // Declaration of static member
public:
    Test() {
        count++;
        id = count;
    }
    void showId() {
        std::cout << "Object ID: " << id << std::endl;
    }
    static int getCount() { // Static member function
        return count; // Can only access static variables
    }
};

// Definition and initialization outside class (mandatory)
int Test::count = 0;

int main() {
    Test t1, t2;
    t1.showId(); // Object ID: 1
    t2.showId(); // Object ID: 2
    std::cout << "Total Objects: " << Test::getCount() << std::endl; // 2
    return 0;
}
```

3. Friend Functions and Friend Classes

To enforce data hiding, non-member functions are strictly blocked from accessing private class attributes. Friendships offer a controlled bypass.

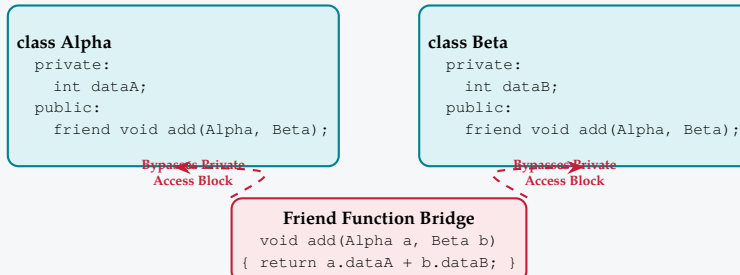
3.1 Friend Functions

Friend Function: A non-member function of a class that is granted permission to access the class's `private` and `protected` members. It is declared inside the class definition with the `friend` keyword.

Special Characteristics of Friend Functions:

- a. It is **not in the scope** of the class to which it has been declared a friend.
- b. It cannot be called using an object of that class (e.g., `obj.friendFunc()` is invalid). It is called like a normal global function: `friendFunc(obj)`.
- c. It usually takes class objects as arguments to process their private fields.
- d. It can be declared in either the `private` or `public` section of the class without altering its behavior.

Friend Function Bridge Bypassing Class Access Restrictions



Code Example: Friend Function accessing private variables of two classes

```
#include <iostream>
class Beta; // Forward declaration required for Alpha's friend prototype

class Alpha {
private:
    int valA;
public:
    Alpha(int v) : valA(v) {}
    friend int add(Alpha, Beta); // Declare friend function
};

class Beta {
private:
    int valB;
public:
    Beta(int v) : valB(v) {}
    friend int add(Alpha, Beta); // Declare same function as friend
};

// Friend function definition (no 'friend' keyword or scope resolution here)
int add(Alpha a, Beta b) {
    return a.valA + b.valB; // Directly accessing private members
}

int main() {
    Alpha a(15);
    Beta b(25);
    std::cout << "Sum = " << add(a, b); // Outputs: Sum = 40
    return 0;
}
```

3.2 Friend Classes

A class can also be declared as a friend of another class. If `class B` is declared as a friend inside `class A`, then **all** member functions of `class B` gain access to the private and protected data members of `class A`.

```
class A { friend class B; }; // Class B is a friend of Class A (9)
```

4. Object Memory and Dynamic Allocation

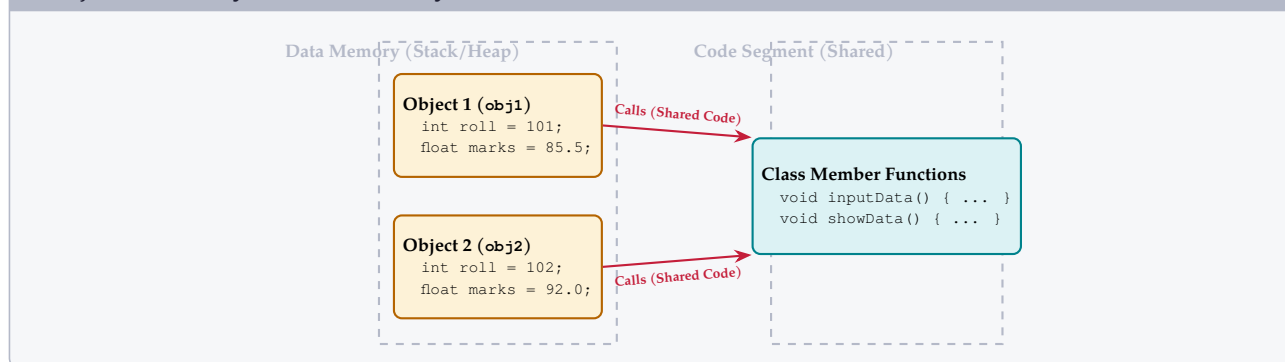
To write efficient code, engineers must understand how objects occupy physical memory and how dynamic memory operators allocate resources on the heap.

4.1 Memory Considerations for Objects

When a class is declared, no memory is allocated. Memory is only allocated when objects of that class are instantiated.

- **Data Members:** Each object receives its own independent memory allocation for non-static data members (to hold distinct object states).
- **Member Functions:** Member functions are allocated only once in the **Code Segment** when the class is loaded. All objects of the class share the exact same memory space for functions. This prevents code redundancy.

Object Memory Allocation Layout in RAM



4.2 Dynamic Memory Management: `new` and `delete`

In addition to C's run-time allocation functions (`malloc/free`), C++ introduces type-safe operators `new` and `delete` which operate directly on the Heap memory.

Dynamic Memory Syntax

Allocate Single Variable:	<code>pointer_var = new data_type;</code>	(10)
Allocate Array of Elements:	<code>pointer_var = new data_type[size];</code>	(11)
Deallocate Single Variable:	<code>delete pointer_var;</code>	(12)
Deallocate Array:	<code>delete [] pointer_var;</code>	(13)

Advantages of `new/delete` over `malloc/free`:

1. **Constructor Calling:** `new` automatically calls the object's constructor; `malloc` only allocates raw bytes.
2. **Destructor Calling:** `delete` automatically calls the destructor before freeing memory; `free` does not.
3. **Type Safety:** `new` returns the exact pointer type (no explicit typecasting needed); `malloc` returns a `void*`.
4. **Safety:** `new` throws a `std::bad_alloc` exception if allocation fails; `malloc` returns `NULL`.

Code Example: Dynamic allocation of variable and array

```
#include <iostream>
int main() {
    // Dynamic single variable allocation
    int *pInt = new int(42); // Allocates int and initializes to 42
    std::cout << "Value: " << *pInt << std::endl;
    delete pInt; // Free memory

    // Dynamic array allocation
    int size = 5;
    int *arr = new int[size]; // Allocates array of 5 integers
    for (int i = 0; i < size; i++) {
        arr[i] = (i + 1) * 10;
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    delete[] arr; // Free array memory (note the square brackets)
    return 0;
}
```

Quick Revision — Module III

Objects and Classes | OE-EC604C

Key Syntax and Configurations

Class Declaration	<code>class MyClass { private: int x; public: void func(); };</code>
External Definition	<code>void MyClass::func() { /* code */ }</code>
Static Definition	<code>int MyClass::staticVar = 0; // Out-of-class definition</code>
Friend Declaration	<code>friend void myFriend(MyClass &obj); // Inside class</code>
Dynamic Single	<code>int *ptr = new int(10); delete ptr;</code>
Dynamic Array	<code>int *arr = new int[5]; delete [] arr;</code>

One-Line Definitions

- **Class:** A user-defined data type that binds data and functions together into a single unit.
- **Object:** An instance of a class that represents a real-world entity and occupies memory.
- **Access Specifiers:** Keywords (`private`, `public`, `protected`) that define the accessibility of class members.
- **Static Data Member:** A class variable shared by all class objects, residing in global static memory.
- **Static Member Function:** A function that can only access static members and is called without an object.
- **Friend Function:** A non-member function declared inside a class that can access its private and protected members.
- **Friend Class:** A class whose member functions can all access the private and protected members of another class.
- **new Operator:** A C++ operator used to dynamically allocate memory on the heap and run constructors.
- **delete Operator:** A C++ operator used to release dynamically allocated heap memory and run destructors.

Frequently Asked / Viva Questions

Q1. What is the default access specifier for class members in C++?

The default access specifier is `private`. If no access specifier is mentioned, members are treated as private.

Frequently Asked / Viva Questions (contd.)**Q2. Why do member functions defined inside the class run faster than those defined outside?**

Member functions defined inside the class are implicitly treated as `inline`, meaning the compiler attempts to substitute the call site with the function code, avoiding call overhead.

Q3. Why must static data members be defined outside the class scope?

The class declaration only defines the type, it does not allocate memory. Defining it outside allocates physical storage in global memory.

Q4. Can a static member function access non-static variables of the class?

No. A static function is not bound to any object instance, so it lacks a `this` pointer and cannot determine which object's non-static variable to access.

Q5. Explain why a friend function cannot be called using the object dot (.) operator.

A friend function is not a member of the class. It is a normal global function with special access rights, so calling it with `obj.` causes a compile error.

Q6. Does declaring a friend function inside a class violate encapsulation?

Theoretically yes, as it bypasses private access blocks. However, it is controlled by the class designer and is useful for operations like operator overloading.

Q7. What is the memory size of an empty class object in C++?

It is 1 byte. The compiler allocates 1 byte to ensure that two different object instances of the empty class have unique memory addresses.

Q8. What happens if you use `delete` instead of `delete []` to free a dynamically allocated array?

It results in undefined behavior. Typically, the compiler will only call the destructor for the first element in the array and may fail to release the entire block of memory.

Q9. What is the difference between class and struct in C++?

In a `struct`, members are `public` by default, whereas in a `class`, members are `private` by default.

Q10. Can we declare a constructor static in C++?

No. Constructors are called to initialize specific object instances. A static function has no instance binding, making a static constructor logically impossible.

Q11. Does `new` return a void pointer like `malloc`?

No. The `new` operator is type-safe and returns a typed pointer to the allocated memory, eliminating the need for casting.

Q12. Can we overload a friend function?

Yes, a friend function is a regular function and can be overloaded like any other non-member function.

Comparison: C++ dynamic operators vs. C memory functions

Feature	new / delete (C++)	malloc / free (C)
Type Safety	Type-safe; returns exact type pointer.	Returns <code>void*</code> ; requires explicit typecasting.
Constructors	Calls constructors automatically for classes.	Does not trigger constructors (allocates raw bytes).
Destructors	Calls destructors automatically.	Does not trigger destructors.
Failure Behavior	Throws <code>std::bad_alloc</code> exception.	Returns <code>NULL</code> .
Buffer Size	Automatically calculated by compiler.	Must be explicitly calculated in bytes using <code>sizeof</code> .

Module 4

Fundamentals of Constructors

1. Fundamentals of Constructors

Constructors are special member functions of a class designed specifically to initialize objects during instantiation.

1.1 Rules and Characteristics

A constructor must follow strict compiler rules:

- **Naming:** It must have the exact same name as the class.
- **Return Type:** It does not have any return type, not even `void`.
- **Execution:** It is automatically invoked by the system when an object of the class is created.
- **Implicit/Explicit Invocation:**

Implicit call: `Complex c1(5, 10);` (14)

Explicit call: `Complex c1 = Complex(5, 10);` (15)

- **Restrictions:** It cannot be declared `virtual`, `static`, or `const`.

1.2 Types of Constructors

1. **Default Constructor:** A constructor that accepts no arguments. If no constructor is written, the compiler automatically generates a default constructor.
2. **Parameterized Constructor:** A constructor that accepts arguments, allowing objects to be initialized with custom values at creation time.
3. **Copy Constructor:** Used to initialize an object using values of another pre-existing object of the same class.
4. **Dynamic Constructor:** A constructor that uses the `new` operator to allocate dynamic heap memory for members during object creation.

2. Deep Copying and Shallow Copying

Copy operations in C++ can lead to critical memory bugs if pointers are involved.

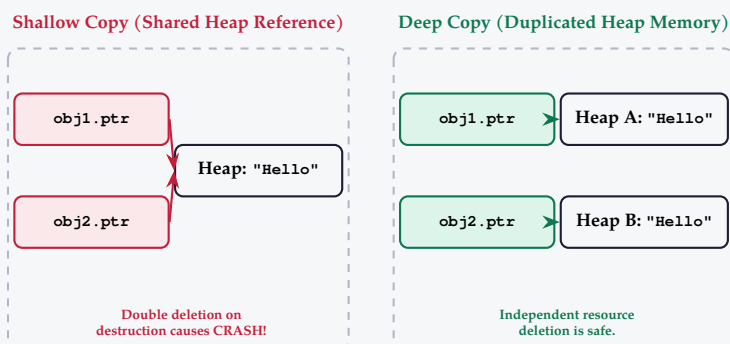
2.1 Shallow Copy vs. Deep Copy

- **Shallow Copy:** The compiler copies member variables bit-by-bit. If a class contains pointers to heap memory, only the address is copied. As a result, both the original and copied objects point to the same memory block.

Shallow Copy Risk: When either object is destroyed, its destructor runs and frees the shared heap memory. The remaining object is left with a dangling pointer, and attempting to delete it results in a double-free crash.

- **Deep Copy:** A user-defined copy constructor allocates a separate block of heap memory and copies the actual values from the source buffer. This isolates the memory blocks.

Memory Block Mappings: Shallow Copy vs. Deep Copy



Code Example: String copy constructor (Deep Copy)

```

#include <iostream>
#include <cstring>
class String {
private:
    char *str;
    int len;
public:
    String(const char *s) {
        len = strlen(s);
        str = new char[len + 1];
        strcpy(str, s);
    }
    // Deep Copy Constructor
    String(const String &source) {
        len = source.len;
        str = new char[len + 1]; // Allocate separate heap memory
        strcpy(str, source.str); // Copy actual data contents
        std::cout << "Copy Constructor Called (Deep Copy)" << std::endl;
    }
    ~String() {
        delete[] str; // Free local pointer safely
    }
    void display() { std::cout << str << std::endl; }
};

```

3. Constructor Overloading & Destructors

C++ allows multiple constructors in a class, selecting the correct one based on arguments. It also ensures proper cleanup when objects go out of scope.

3.1 Constructor Overloading and Default Arguments

Classes can have multiple constructors with different parameter signatures. They can also contain default arguments.

Constructor 1 (Default): `Complex() { real = imag = 0; }`

Constructor 2 (Parameterized): `Complex(double r, double i = 0) { real = r; imag = i; }`

3.2 Destructors

Destructor: A special member function of a class that is called automatically when an object goes out of scope or is explicitly deleted. Its purpose is to release resource allocations (like heap memory, open file handlers, socket connections).

Key Destructor Rules:

- Naming: Same name as class, prefixed with a tilde symbol (~).
- Arguments: Cannot take any parameters or return values. Overloading is not possible.
- Execution Order: Objects are destroyed in the **reverse order** of their creation (Last In, First Out).

Code Example: Tracing Constructor and Destructor Executions

```
#include <iostream>
class Trace {
private:
    int id;
public:
    Trace(int val) : id(val) {
        std::cout << "Constructor called for " << id << std::endl;
    }
    ~Trace() {
        std::cout << "Destructor called for " << id << std::endl;
    }
};

void runLocal() {
    std::cout << "--- Enter local scope ---" << std::endl;
    Trace localObj(2);
    std::cout << "--- Exit local scope ---" << std::endl;
}

int main() {
    std::cout << "--- Main Start ---" << std::endl;
    Trace globalObj(1);
    runLocal();
    std::cout << "--- Main End ---" << std::endl;
    return 0;
}
```

The code above outputs:

```
--- Main Start ---
Constructor called for 1
--- Enter local scope ---
Constructor called for 2
--- Exit local scope ---
Destructor called for 2
--- Main End ---
Destructor called for 1
```

Quick Revision — Module IV

Constructors and Destructors | OE-EC604C

Key Constructor Syntax

Default Constructor	<code>MyClass() { x = 0; }</code>
Parameterized	<code>MyClass(int val) : x(val) { } // uses initializer list</code>
Copy Constructor	<code>MyClass(const MyClass &src) { x = src.x; } // reference is mandatory</code>
Destructor	<code>~MyClass() { /* release heap memory / close files */ }</code>

One-Line Definitions

- **Constructor:** A special member function with the class name that initializes objects upon instantiation.
- **Default Constructor:** A constructor that takes no parameters (either empty or with all arguments defaulted).
- **Copy Constructor:** A constructor used to create an object as a clone of an existing object of the same class.
- **Dynamic Constructor:** A constructor that allocates heap memory dynamically during object initialization.
- **Destructor:** A special member function prefixed with ~ that deallocates resources when an object is destroyed.
- **Shallow Copy:** A member-by-member copy of an object, copying pointer addresses rather than the target data.
- **Deep Copy:** A copy operation that duplicates the heap resources pointed to by an object, creating an independent clone.
- **Initializer List:** A comma-separated list prefixing a constructor body used to initialize data members directly.
- **Double Deletion:** A crash state occurring when two objects point to the same heap block and both attempt to delete it.

Frequently Asked / Viva Questions

Q1. Why is it mandatory to pass the argument by reference in a copy constructor?

If passed by value, the function call itself would invoke the copy constructor to clone the parameter, causing infinite recursion until the stack overflows. E.g., `MyClass(MyClass source)` is incorrect.

Frequently Asked / Viva Questions (contd.)**Q2. Can a class have more than one destructor?**

No. Destructors cannot accept arguments, which means they cannot be overloaded. Every class has exactly one destructor.

Q3. Under what conditions does the C++ compiler write a default copy constructor for you?

If a class does not define any copy constructor, the compiler automatically generates a default copy constructor, which performs a shallow copy.

Q4. Explain the order of execution of constructors and destructors in hybrid structures.

Constructors are invoked in the order of inheritance (base classes first, then derived classes). Destructors run in the exact reverse order.

Q5. Is it possible to declare a private constructor in C++?

Yes. A private constructor prevents objects from being created outside the class scope. This pattern is commonly used in the Singleton Design Pattern.

Q6. What is the difference between copy constructor and assignment operator (=)?

The copy constructor initializes a newly created object using a source object (e.g., `MyClass a = b;`). The assignment operator copies values to an already existing object (e.g., `a = b;`).

Q7. Does a destructor release the memory of the object itself?

No. A destructor only frees auxiliary resources allocated *inside* the object (e.g., heap pointers, file descriptors). The memory for the object structure itself is freed by the stack or heap manager.

Q8. Why can't we declare a constructor as `virtual`?

Virtual functions rely on the V-Table (virtual table) pointer inside an object. During construction, the object is not fully formed, and the V-Ptr is not yet initialized. Thus, constructors cannot be virtual.

Q9. What is the purpose of declaring copy constructor arguments as `const`?

Declaring the argument as `const` (e.g., `const MyClass &obj`) prevents the copy constructor from accidentally modifying the source object.

Q10. What is a member initializer list and when is it mandatory?

It is a constructor mechanism used to initialize members directly. It is mandatory for: (1) initializing reference variables, (2) initializing `const` members, and (3) calling parameterized base class constructors.

Q11. Can we call a constructor explicitly like a normal function?

Yes, we can call it explicitly (e.g., `Complex c = Complex(1.0, 2.0);`), which creates a temporary object that is then copied or moved.

Frequently Asked / Viva Questions (contd.)

Q12. What happens if an exception is thrown inside a constructor?

If an exception is thrown in a constructor, the object is considered only partially constructed, and its destructor will not run. This can lead to memory leaks unless resources are managed via smart pointers.

Comparison: Shallow Copy vs. Deep Copy

Feature	Shallow Copy	Deep Copy
Heap Memory	Shares the existing heap memory between objects.	Allocates new, independent memory blocks on the heap.
Pointer Safety	Pointer address is copied. Vulnerable to dangling pointers.	Pointers point to separate blocks. Safe from cross-corruption.
Crash Risk	High. Double deletion in destructors crashes the program.	Safe. Destructors free distinct blocks independently.
Default Behavior	Executed by default compiler copy constructors.	Must be manually coded by overriding the copy constructor.
Execution Speed	Extremely fast (simple bitwise copy).	Slightly slower (calls heap allocator and copies bytes).

Module 5

Fundamentals of Operator Overloading

1. Fundamentals of Operator Overloading

Operator overloading is a feature of compile-time polymorphism that allows user-defined types (classes) to interact using standard C++ operators.

1.1 General Rules and Syntax

When overloading operators in C++, the following rules must be observed:

- Only existing operators can be overloaded. New operators (like `**` or `<>`) cannot be created.
- The precedence and associativity of operators remain unchanged.
- Overloaded operators must have at least one operand of user-defined type (class/enum).
- Original syntax templates (e.g., number of operands) must be maintained.

List of Operators that Cannot be Overloaded

The following operators are strictly restricted from being overloaded:

1. Class member access operator (`.`)
2. Pointer-to-member operator (`.*`)
3. Scope resolution operator (`::`)
4. Ternary conditional operator (`?:`)
5. Sizeof operator (`sizeof`)

Operator Overloading Signature Syntax

As Member Function: `Return_Type operator OP (Arguments) {...}` (16)

As Friend Function: `friend Return_Type operator OP (Arguments) {...}` (17)

2. Binary & Unary Operator Overloading

Operators are overloaded differently based on whether they are unary (one operand) or binary (two operands), and whether member or friend functions are used.

2.1 Unary Operator Overloading

Unary operators operate on a single operand.

- **Member Function implementation:** Takes **zero** arguments (the operand is implicitly passed via the `this` pointer).
- **Friend Function implementation:** Takes **one** argument explicitly.
- **Increment/Decrement Ambiguity:** To distinguish between prefix (e.g., `++obj`) and postfix (e.g., `obj++`), C++ uses a dummy `int` parameter in the postfix signature.

Code Example: Prefix and Postfix operator overloading

```
#include <iostream>
class Counter {
private:
    int value;
public:
    Counter(int v = 0) : value(v) {}

    // Prefix ++ (e.g., ++c)
    Counter operator++() {
        ++value;
        return *this; // Returns incremented object
    }

    // Postfix ++ (e.g., c++) -> Note the dummy 'int' parameter
    Counter operator++(int) {
        Counter temp = *this; // Save original state
        value++; // Increment
        return temp; // Return original state
    }

    int getValue() { return value; }
};
```

2.2 Binary Operator Overloading

Binary operators operate on two operands (e.g., `a + b`).

- **Member Function implementation:** Takes **one** argument explicitly. The left-hand operand is the calling object, and the right-hand operand is passed as the argument.
- **Friend Function implementation:** Takes **two** arguments explicitly.

Code Example: Relational operator overloading for custom strings

```

#include <iostream>
#include <cstring>
class String {
private:
    char *str;
public:
    String(const char *s) {
        str = new char[strlen(s) + 1];
        strcpy(str, s);
    }
    ~String() { delete[] str; }

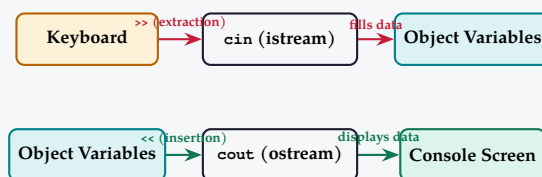
    // Overload == operator
    bool operator==(const String &s) {
        return strcmp(str, s.str) == 0;
    }

    // Overload < operator
    bool operator<(const String &s) {
        return strcmp(str, s.str) < 0;
    }
};

```

2.3 Stream Insertion (<<) and Extraction (>>) Overloading

Stream operators must be overloaded as **friend functions** because the left-hand operand is a stream object (like `std::ostream`), not an instance of our class.

Stream I/O Data Flow Block Diagram**3. Type Conversions**

C++ allows custom type conversions between primitive types and class objects.

3.1 Basic Type to Class Type

Handled automatically using a **parameterized constructor** inside the target class that accepts the basic type as an argument.

```
Time t1 = 95; // Invokes constructor Time(int) to convert minutes (18)
```

3.2 Class Type to Basic Type

Accomplished by defining a **casting operator function** (conversion function) inside the class.

Rules for Conversion Functions:

- It must be a class member function.
- It must not specify any return type (the return type is implicit from the operator's name).
- It must not accept any arguments.

Syntax: `operator Basic_Type () { return value; }` (19)

Code Example: Basic-to-Class and Class-to-Basic conversions

```
#include <iostream>
class Time {
private:
    int hours, minutes;
public:
    Time() : hours(0), minutes(0) {}

    // Basic to Class Conversion (Constructor)
    Time(int mins) {
        hours = mins / 60;
        minutes = mins % 60;
    }

    // Class to Basic Conversion (Casting Operator)
    operator int() {
        return (hours * 60) + minutes;
    }

    void display() {
        std::cout << hours << "h " << minutes << "m" << std::endl;
    }
};

int main() {
    // Basic -> Class
    Time t1 = 85; // Converts 85 (int) to 1h 25m (Time)
    t1.display(); // Prints: 1h 25m

    // Class -> Basic
    int totalMins = t1; // Converts Time to int using operator int()
    std::cout << "Total minutes = " << totalMins << std::endl; // 85
    return 0;
}
```

Quick Revision — Module V

Operator Overloading | OE-EC604C

Key Overloading Signatures

Unary (Member)	<code>Counter operator++() { /* prefix */ }</code>
Unary Postfix (Member)	<code>Counter operator++(int) { /* dummy int parameter */ }</code>
Binary (Member)	<code>Complex operator+(const Complex &c) { /* 1 arg */ }</code>
Binary (Friend)	<code>friend Complex operator+(Complex &c1, Complex &c2); // 2 args</code>
Stream Insertion	<code>friend ostream& operator<<(ostream& out, const MyClass &obj);</code>
Casting Function	<code>operator float() { return data; } // No return type specified</code>

One-Line Definitions

- **Operator Overloading:** Assigning new functionalities to C++ operators when applied to class instances.
- **Stream Extraction (»):** An overloaded operator used to extract formatted values from an input stream.
- **Stream Insertion («):** An overloaded operator used to insert formatted values into an output stream.
- **Conversion Operator:** A special class member function used to convert an object into another data type.
- **Prefix Operator:** An increment or decrement operator executed before evaluation of the expression.
- **Postfix Operator:** An increment or decrement operator executed after evaluation of the expression.
- **Basic to Class Conversion:** Converting standard types to user classes via parameterized constructors.
- **Class to Basic Conversion:** Converting classes to standard types via overloaded casting operators.
- **This Pointer:** An implicit pointer inside member functions that points to the calling object instance.

Frequently Asked / Viva Questions**Q1. Which operators cannot be overloaded in C++? Why?**

Operators like `.`, `.*`, `::`, `?:`, and `sizeof` cannot be overloaded to protect C++ syntax structures and core language bindings from corruption.

Q2. Why does a postfix overloaded operator require a dummy `int` parameter?

The compiler uses the `int` parameter strictly as a signature flag to distinguish postfix calls (`obj++`) from prefix calls (`++obj`).

Q3. Why must stream insertion (`<<`) and extraction (`>>`) operators be overloaded as friend functions?

The left-hand operand is `ostream` or `istream`, not our class object. Member functions require the left operand to be the class itself, so a friend function is mandatory.

Q4. What is the difference in arguments between member and friend function operator overloading?

For binary operators: member functions take 1 argument (left operand is implicit); friend functions take 2 arguments explicitly. For unary operators: member functions take 0 arguments; friend functions take 1.

Q5. Can we change the basic behaviors of operators (like changing `+` to subtract)?

Yes, syntactically possible, but highly discouraged as it violates design clarity and makes code unreadable.

Q6. What are the restrictions on type conversion functions?

They must be non-static class member functions, cannot have arguments, and must not specify a return type.

Q7. Explain conversion from one class type to another class type.

This can be done using: (1) A constructor in the destination class that accepts the source class, or (2) A casting operator inside the source class that returns the destination class.

Q8. Can we overload the assignment operator (`=`) as a friend function?

No. C++ rules mandate that `=`, `()`, `[]`, and `->` must be overloaded *only* as member functions to prevent compiler syntax conflicts.

Q9. What is the return type of stream insertion (`<<`) and extraction (`>>`) operators?

They return a reference to the stream object (`ostream&` or `istream&`). This allows chaining (e.g., `cout << a << b;`).

Q10. Does operator overloading change the execution speed of a program?

No. Operator overloading is resolved at compile time (static binding), so it has zero runtime overhead compared to equivalent function calls.

Q11. Is it possible to overload operators for standard types (like overloading `+` for two integers)?

Frequently Asked / Viva Questions (contd.)

No. At least one operand must be a user-defined class or structure to prevent overriding standard arithmetic logic.

Q12. Can we use default arguments in overloaded operators?

No, default arguments are not permitted in overloaded operators because they could conflict with the operator's expected arity.

Comparison: Member Functions vs. Friend Functions for Overloading

Feature	Member Function Overloading	Friend Function Overloading
Implicit Operand	Passes the left-hand operand implicitly via the <code>this</code> pointer.	No implicit operand. All operands must be explicitly declared in arguments.
Left Operand Rule	Left operand must strictly be an object of the defining class.	Left operand can be of any type (e.g., stream, primitive, other class).
Number of Arguments	Binary: 1 argument; Unary: 0 arguments.	Binary: 2 arguments; Unary: 1 argument.
Class Encapsulation	Maintains strict encapsulation boundaries.	Grants external function access (minor encapsulation bypass).
Special Operators	Mandatory for operators: <code>=</code> , <code>[]</code> , <code>()</code> , <code>-></code> .	Not allowed for: <code>=</code> , <code>[]</code> , <code>()</code> , <code>-></code> .

Module 6

Fundamentals of Inheritance & Derivation Levels

1. Fundamentals of Inheritance & Derivation Levels

Inheritance is the mechanism by which a new class (derived class) is built from an existing class (base class), inheriting its properties and behaviors. This promotes code reusability.

1.1 Base and Derived Classes

A derived class inherits members of a base class. The visibility of inherited members depends on the **inheritance access level** declared:

1. Public Derivation (`class Derived : public Base`):

- Public base members become **public** in the derived class.
- Protected base members become **protected** in the derived class.
- Private base members are **never accessible** directly (can only be accessed via public/protected base functions).

2. Protected Derivation (`class Derived : protected Base`):

- Public and protected base members become **protected** in the derived class.

3. Private Derivation (`class Derived : private Base`):

- Public and protected base members become **private** in the derived class.

1.2 Types of Inheritance

C++ supports five distinct inheritance topologies:

- **Single Inheritance:** One base class, one derived class.
- **Multilevel Inheritance:** A class is derived from another derived class (e.g., A → B → C).
- **Multiple Inheritance:** A derived class has more than one base class (e.g., A and B inherited by C).
- **Hierarchical Inheritance:** Multiple derived classes inherit from a single base class.
- **Hybrid Inheritance:** A combination of two or more inheritance types (e.g., multiple + hierarchical).

2. Constructor & Destructor Execution Sequence

When a derived class object is instantiated, constructors are executed along the inheritance chain to ensure that the entire object is initialized.

2.1 Invocation Sequence Rules

- **Constructors:** Executed from **top to bottom** along the class hierarchy. The base class constructor is executed first, followed by the derived class constructor.
- **Destructors:** Executed in the **exact reverse order** (bottom to top).
- **Initializer Lists:** Parameterized base constructors must be explicitly called using the derived class constructor's member initialization list.

Code Example: Invocation of parameterized base constructors

```
#include <iostream>
class Base {
protected:
    int baseVal;
public:
    Base(int val) : baseVal(val) {
        std::cout << "Base Constructor: " << baseVal << std::endl;
    }
    ~Base() {
        std::cout << "Base Destructor" << std::endl;
    }
};

class Derived : public Base {
private:
    int derivedVal;
public:
    // Call base constructor explicitly in initialization list
    Derived(int bVal, int dVal) : Base(bVal), derivedVal(dVal) {
        std::cout << "Derived Constructor: " << derivedVal << std::endl;
    }
    ~Derived() {
        std::cout << "Derived Destructor" << std::endl;
    }
};
```

3. The Diamond Inheritance Ambiguity

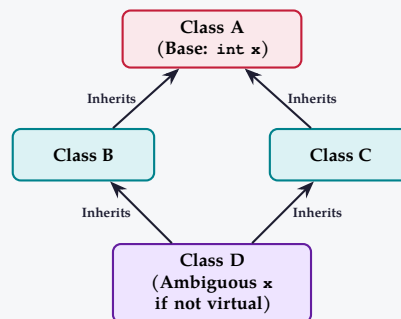
Multipath inheritance can result in a duplicate copy conflict known as the **Diamond Problem**.

3.1 The Diamond Problem and Virtual Base Classes

Consider class `D` inheriting from classes `B` and `C`, which both inherit from class `A`. If `A` contains a member `x`, `D` receives two separate copies of `x` (one via `B` and one via `C`). Any reference to `x` inside `D` is flagged as **ambiguous** by the compiler.

Virtual Base Class: By declaring the base class `A` as `virtual` during derivation in `B` and `C`, the compiler ensures that only one shared instance of class `A` is created in memory, resolving all inheritance path ambiguities.

Diamond Inheritance Path Topologies



Without `virtual`: `D` has duplicate copies of `A`'s members.
With `virtual`: `D` shares a single, resolved instance of `A`.

Code Example: Resolving Diamond Problem using Virtual Base Class

```
#include <iostream>
class A {
public:
    int x;
};

// Declare B and C with virtual public inheritance
class B : virtual public A {
public:
    int y;
};

class C : virtual public A {
public:
    int z;
};

class D : public B, public C {
public:
    int sum;
    void show() {
        x = 10; // Ambiguous without virtual keyword! Now resolved.
        y = 20;
        z = 30;
        sum = x + y + z;
        std::cout << "Sum = " << sum << std::endl;
    }
};
```

4. Containership vs. Inheritance

To design clean systems, software developers must choose the correct structural relationship between classes:

- **Inheritance (is-a relationship):** A derived class inherits properties of the base class. E.g., a Car **is a** Vehicle.
- **Containership (has-a relationship):** An object of one class is embedded as a data member inside another class. E.g., a Car **has an** Engine.

Quick Revision — Module VI

Inheritance | OE-EC604C

Access Control Table under Derivation

Base Member	Public Derivation	Protected Derivation	Private Derivation
Private	Not Accessible	Not Accessible	Not Accessible
Protected	Protected in Derived	Protected in Derived	Private in Derived
Public	Public in Derived	Protected in Derived	Private in Derived

One-Line Definitions

- **Inheritance:** The mechanism of deriving a new class from an existing base class to reuse code.
- **Base Class:** The existing class from which features are inherited.
- **Derived Class:** The new class that inherits features from one or more base classes.
- **Diamond Problem:** An ambiguity issue where a derived class inherits duplicate members of a grandparent class through two distinct paths.
- **Virtual Base Class:** A base class declared virtual to ensure only one instance is shared in multipath inheritance.
- **Containership:** Embedding an object of one class as a member variable inside another class (Composition).
- **Initialization List (Base):** The derived constructor syntax used to pass arguments to base class constructors.
- **Is-A Relationship:** Logical relationship modeled by class inheritance (e.g., Dog is-a Mammal).
- **Has-A Relationship:** Logical relationship modeled by class containership (e.g., Dog has-a Tail).

Frequently Asked / Viva Questions

Q1. Why are private base class members not accessible in derived classes?

This maintains the privacy boundaries of class design. If derived classes could access private members directly, encapsulation could be easily broken.

Q2. Explain why virtual inheritance is necessary to solve the Diamond Problem.

Virtual inheritance directs the compiler to build only one shared instance of the grandparent class, resolving lookup ambiguity for its members.

Q3. What is the execution order of constructors in multiple inheritance?

Base constructors are executed in the order they are listed in the inheritance

Frequently Asked / Viva Questions (contd.)

declaration, NOT the order they appear in the initializer list. E.g., `class D : public B, public C` calls B's constructor first, then C's.

Q4. Which class constructor is executed first in multilevel inheritance?

The absolute root base class constructor is executed first, followed by intermediate bases, ending at the leaf derived class.

Q5. Does a derived class inherit the constructor of the base class?

No. Constructors are class-specific and are not inherited. A derived class must define its own constructors or use the default constructor.

Q6. Can we declare a destructor virtual? Why?

Yes, virtual destructors are mandatory when deleting derived objects via base pointers to ensure derived resource cleanup.

Q7. What is the difference between multiple inheritance and hybrid inheritance?

Multiple inheritance involves deriving from 2+ distinct base classes. Hybrid inheritance combines multiple topologic models (like multiple and multilevel).

Q8. How does containership differ from private inheritance?

Containership embeds objects as members, restricting them to public APIs. Private inheritance makes base members private, permitting code access but hiding public APIs from external callers.

Q9. What happens if base and derived classes declare members with the same name?

The derived member hides the base member. The base member can still be accessed using the scope resolution operator: `BaseClass::memberName`.

Q10. Can a class inherit from itself in C++?

No, circular or self-inheritance (e.g., `class A : public A`) is syntactically invalid and causes compile errors.

Q11. Why does virtual inheritance require base class constructors to be called by the most derived class?

Since the base class instance is shared, intermediate derived classes cannot initialize it independently. The most derived class constructor executes its initialization.

Q12. How does the visibility level of inheritance affect derived class inheritance hierarchies?

Private inheritance prevents downstream subclasses from accessing grandparent class members, whereas public/protected inheritances allow subclass access.

Comparison: Multiple Inheritance vs. Hybrid Inheritance

Feature	Multiple Inheritance	Hybrid Inheritance
Structure	A single subclass derives directly from two or more parent classes.	Combines two or more basic inheritance shapes (e.g., hierarchic, multilevel, multiple).
Diamond Risk	None, unless parent classes share a common grandparent class.	High. Multipath inheritance frequently creates diamond path ambiguities.
Complexity	Moderate. Simple to resolve using qualifiers.	High. Requires virtual inheritance declaration to resolve duplicate members.
Design Goal	Merging features from multiple independent source interfaces.	Modeling complex real-world classification hierarchies.

Module 7

Fundamentals of Polymorphism

1. Fundamentals of Polymorphism

Polymorphism ("many forms") allows a single interface to represent different underlying forms (behaviors). In C++, it is broadly classified into compile-time and run-time polymorphism.

1.1 Pointers to Objects and Polymorphic Assignments

Pointers can store the addresses of objects. In inheritance hierarchies, C++ supports upcasting:

Upcasting Rule: A base class pointer can point to a derived class object without explicit typecasting. However, using this pointer, we can only access base class members, unless the functions are virtual.

```
Base* ptr = new Derived(); // Polymorphic Assignment (20)
```

1.2 The `this` Pointer

The `this` pointer is an implicit parameter passed to all non-static member functions. **Characteristics:**

- It stores the address of the calling object.
- Used to resolve naming conflicts between member variables and parameters: `this->data = data;`
- Used to return the calling object itself to enable **method chaining**: `return *this;`

2. Compile-Time vs. Run-Time Polymorphism

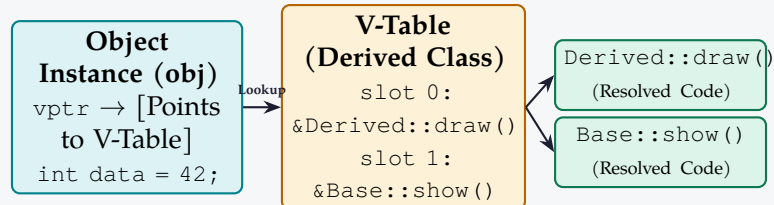
The primary distinction lies in when the compiler resolves function invocations.

2.1 Virtual Functions and Late Binding

By prefixing a base class function declaration with the `virtual` keyword, we tell the compiler to resolve function calls at **run time** (late binding) based on the type of the object pointed to, rather than the type of the pointer.

Mechanism of Late Binding (V-Table and V-Ptr):

1. **V-Ptr (Virtual Pointer):** Every object of a class containing virtual functions is automatically allocated a hidden pointer field (`vp_ptr`) pointing to its class's V-Table.
2. **V-Table (Virtual Method Table):** A static table compiled for each class containing function addresses for all its virtual functions.

Virtual Method Table (V-Table) Mapping Layout

Code Example: Runtime polymorphism using base pointer arrays

```
#include <iostream>
class Shape {
public:
    virtual void draw() { // Virtual function
        std::cout << "Drawing generic shape" << std::endl;
    }
    virtual ~Shape() {} // Virtual destructor
};

class Circle : public Shape {
public:
    void draw() override { // Overridden function
        std::cout << "Drawing Circle" << std::endl;
    }
};

class Box : public Shape {
public:
    void draw() override {
        std::cout << "Drawing Box" << std::endl;
    }
};

int main() {
    Shape* shapes[2];
    shapes[0] = new Circle();
    shapes[1] = new Box();

    for(int i = 0; i < 2; i++) {
        shapes[i]->draw(); // Late binding: calls appropriate draw()
        delete shapes[i];
    }
    return 0;
}
```

3. Pure Virtual Functions and Abstract Classes

Sometimes, a base class is so generic that it cannot provide any logical default implementation for its virtual functions.

3.1 Pure Virtual Functions

Pure Virtual Function: A virtual function declared in a base class with no implementation, represented by assigning = 0 at its declaration.

Declaration: `virtual Return_Type FuncName(Args) = 0;` (21)

3.2 Abstract Classes

A class containing at least one pure virtual function is an **Abstract Class**. **Key Characteristics:**

- **Instantiation:** Abstract classes cannot be used to instantiate objects. E.g., `Employee emp;` is a compile error.
- **Subclassing:** Derived classes must override **all** inherited pure virtual functions to become concrete classes. Failure to do so makes the derived class abstract as well.

Code Example: Abstract Employee base class

```
#include <iostream>
#include <string>
class Employee {
protected:
    std::string name;
public:
    Employee(std::string n) : name(n) {}
    virtual double calculateSalary() = 0; // Pure virtual function
    virtual void display() {
        std::cout << "Employee: " << name << std::endl;
    }
    virtual ~Employee() {}
};

class FullTimeEmployee : public Employee {
private:
    double monthlySalary;
public:
    FullTimeEmployee(std::string n, double s) : Employee(n), monthlySalary(s) {}
    double calculateSalary() override { return monthlySalary; }
};
```

4. Virtual Destructors & Memory Leaks

When destroying dynamically allocated derived class objects through base class pointers, memory leaks will occur unless the base class destructor is virtual.

4.1 Destructor Call Failures

If the base class destructor is **non-virtual**, the compiler binds the destructor call to the base pointer type at compile time. As a result, only the base class destructor executes. The derived class destructor never runs, leaving any heap allocations or handles inside the derived class unreleased.

Code Example: Memory leak prevention via Virtual Destructors

```
#include <iostream>
class Base {
public:
    Base() { std::cout << "Base Constructor" << std::endl; }
    // Declaring destructor virtual guarantees derived deallocation
    virtual ~Base() { std::cout << "Base Destructor" << std::endl; }
};

class Derived : public Base {
private:
    int* data;
public:
    Derived() {
        std::cout << "Derived Constructor" << std::endl;
        data = new int[100]; // Heap allocation
    }
    ~Derived() {
        std::cout << "Derived Destructor (Releasing Heap)" << std::endl;
        delete[] data; // Releases resource safely
    }
};

int main() {
    Base* ptr = new Derived();
    delete ptr; // Triggers Derived destructor then Base destructor
    return 0;
}
```

Quick Revision — Module VII

Polymorphism | OE-EC604C

Key Polymorphic Syntax

Virtual Declaration	<code>virtual void print(); // Inside base class declaration</code>
Override Qualifier	<code>void print() override; // Inside derived class (C++11)</code>
Pure Virtual Function	<code>virtual void process() = 0; // Defines abstract interface</code>
Virtual Destructor	<code>virtual ~Base() { /* virtual base destructor */ }</code>

One-Line Definitions

- **Polymorphism:** The ability of different classes to respond uniquely to identical message calls.
- **Virtual Function:** A class member function overridden in derived classes to ensure late binding execution.
- **Late Binding:** Resolving function calls at run time based on the active object's type.
- **V-Table:** A compiler-generated static lookup table storing virtual function addresses for a class.
- **V-Ptr:** An object instance pointer referencing the V-Table allocated during object instantiation.
- **Pure Virtual Function:** An abstract base method initialized to = 0 that derived classes must override.
- **Abstract Class:** A class containing pure virtual methods that cannot be instantiated.
- **Virtual Destructor:** A base class destructor configured to trigger proper intermediate subclass destructor sequences.
- **This Pointer:** An implicit member pointer storing the memory address of the calling object.

Frequently Asked / Viva Questions

Q1. What is the difference between early binding and late binding?

Early binding (compile time) resolves function addresses using pointer types. Late binding (run time) resolves addresses using the pointed-to object type via the V-Table lookup.

Q2. Can we declare a virtual function inside a structure (`struct`)?

Frequently Asked / Viva Questions (contd.)

Yes. Structures in C++ are identical to classes but use public access by default. They support the `virtual` keyword and inheritance.

Q3. What is the size impact of adding virtual functions to a class?

It adds the size of a single pointer (`vptr`) to every object instance (typically 4 bytes on 32-bit and 8 bytes on 64-bit systems), regardless of how many virtual functions are defined.

Q4. Can a class have virtual static member functions?

No. Static member functions are class-wide and do not bind to object instances. Since they lack a `this` pointer and `vptr` links, they cannot be virtual.

Q5. Can we override private virtual functions in C++?

Yes. The derived class can override private base virtual functions. Access levels only restrict calling permissions, not inheritance resolution.

Q6. What is the difference between overloading and overriding?

Overloading occurs within the same class scope (same name, different arguments). Overriding occurs between base and derived classes (same name, identical arguments, uses `virtual`).

Q7. Can an abstract class have constructors?

Yes. An abstract class cannot be instantiated directly, but its constructors run during derived class constructor sequences to initialize inherited data members.

Q8. Why does a constructor declare base classes virtual during multiple derivations?

To bypass duplicate grandparent member allocations in hybrid structures (solving the Diamond Problem).

Q9. What happens if we forget to declare base destructors virtual?

Deleting derived class instances through base class pointers only calls the base destructor, leaking derived heap resources.

Q10. What is a pure abstract class or interface?

A class that contains *only* pure virtual functions and no data members. It acts purely as a behavior contract.

Q11. Can we call a virtual function from a constructor or destructor?

Yes, but early binding is used. It calls the local class's implementation, not the derived class's version, because the derived object is not yet fully constructed or is already destroyed.

Q12. What is the purpose of the `override` keyword in C++11?

It instructs the compiler to verify that the function matches a virtual base class signature, preventing signature mismatch errors.

Comparison: Compile-Time vs. Run-Time Polymorphism

Feature	Compile-Time Polymorphism	Run-Time Polymorphism
Binding Time	resolved at compile time (Static Binding / Early Binding).	resolved at run time (Dynamic Binding / Late Binding).
Mechanism	Function Overloading and Operator Overloading.	Virtual Functions.
Speed	Fast execution (no run-time address lookup overhead).	Slightly slower (requires dereferencing the V-Ptr and V-Table lookup).
Flexibility	Fixed logic flow determined during compilation.	Highly flexible; handles new derived classes at runtime.
Requirements	Unique signatures (number/type of parameters).	Identical signatures; requires base pointer/reference interface.

Module 8

C++ File Streams Hierarchy

1. C++ File Streams Hierarchy

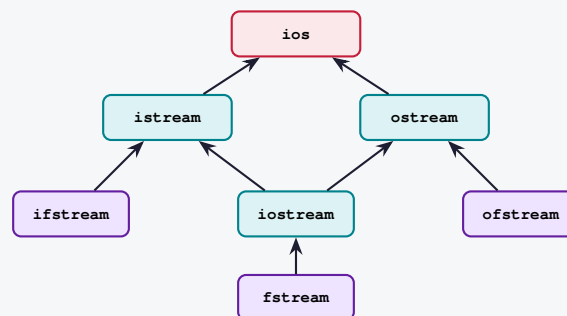
C++ models file input and output operations using a system of classes inherited from basic stream classes.

1.1 The Stream Class Hierarchy

All stream classes are derived from a common virtual base class `ios`. The main classes used for file operations are:

- **ifstream**: Input file stream. Derived from `istream`, used for reading files.
- **ofstream**: Output file stream. Derived from `ostream`, used for creating and writing files.
- **fstream**: Input/Output file stream. Derived from `iostream`, used for simultaneous reading and writing operations.

C++ Stream Class Hierarchy Topology



2. Opening & Closing Files

Files can be opened in two ways: using the constructor or using the `open()` method.

2.1 File Open Methods

1. **Using Constructor:** Best when managing a single file throughout the stream's lifetime.

```
ofstream outFile("data.txt"); // Opens file automatically (22)
```

2. **Using `open()` method:** Best when reusing a single stream object to access multiple files sequentially.

```
ifstream inFile; inFile.open("data.txt");
```

 (23)

2.2 File Modes

File modes specify the access permissions and options for opening files.

File Mode Constant	Description
<code>ios::in</code>	Opens the file for reading (default for <code>ifstream</code>).
<code>ios::out</code>	Opens the file for writing (default for <code>ofstream</code>). Erases existing content.
<code>ios::app</code>	Append mode. All write operations occur at the end of the file.
<code>ios::ate</code>	Opens the file and moves the pointer to the end of the file.
<code>ios::binary</code>	Opens the file in binary mode (instead of default text mode).

3. File Pointers and Seek Operations

Each file stream object maintains file pointers that keep track of reading or writing locations.

3.1 Seek and Tell Functions

- **`seekg()`:** Moves the **get** (read) pointer to a new position.
- **`seekp()`:** Moves the **put** (write) pointer to a new position.
- **`tellg()`:** Returns the current offset position of the get pointer.
- **`tellp()`:** Returns the current offset position of the put pointer.

Seek Syntax Parameters

```
stream.seekg(offset, direction);
```

 (24)

Where direction can be: `ios::beg` or `ios::cur` or `ios::end`

 (25)

File Pointer Position and Offset Directions



4. Binary & Random Access Operations

By default, files are opened in text mode. Binary mode allows reading and writing complete structures or objects to files directly without parsing.

4.1 Object Writing using `write()` and `read()`

```
file.write(reinterpret_cast<char*>(&object), sizeof(object)); (26)
```

```
file.read(reinterpret_cast<char*>(&object), sizeof(object)); (27)
```

Code Example: Reading and writing objects to binary files

```
#include <iostream>
#include <fstream>
#include <cstring>
class Student {
private:
    char name[30];
    int roll;
public:
    Student() : roll(0) { name[0] = '\0'; }
    Student(const char* n, int r) {
        strcpy(name, n);
        roll = r;
    }
    void show() {
        std::cout << "Roll: " << roll << ", Name: " << name << std::endl;
    }
};

void saveRecord() {
    Student s1("Biraj", 101);
    std::ofstream file("student.dat", std::ios::out | std::ios::binary);
    file.write(reinterpret_cast<char*>(&s1), sizeof(s1));
    file.close();
}

void loadRecord() {
    Student s2;
    std::ifstream file("student.dat", std::ios::in | std::ios::binary);
    file.read(reinterpret_cast<char*>(&s2), sizeof(s2));
    s2.show(); // Prints: Roll: 101, Name: Biraj
    file.close();
}
```

4.2 Random Access Database Record Update

Using `seekp()` and index logic, we can update specific object slots directly inside a binary database without reading the whole file.

Code Example: Random Access database record update

```
#include <iostream>
#include <fstream>
struct Account {
    int accNum;
    double balance;
};

void updateBalance(int recordId, double newBalance) {
    std::fstream file("account.dat", std::ios::in | std::ios::out | std::ios::binary);
    Account acc;

    // Jump to the specific record offset using sizeof
    int offset = (recordId - 1) * sizeof(Account);
    file.seekp(offset, std::ios::beg);

    acc.accNum = recordId;
    acc.balance = newBalance;

    // Rewrite record
    file.write(reinterpret_cast<char*>(&acc), sizeof(Account));
    file.close();
}
```

Quick Revision — Module VIII

Files and Streams | OE-EC604C

Key File I/O Syntax

Open Write (Trunc)	<code>ofstream out("a.txt", ios::out); // overrides</code>
Open Append	<code>ofstream out("a.txt", ios::app); // appends</code>
Binary Open	<code>fstream file("a.dat", ios::in ios::out ios::binary);</code>
Seek from Beginning	<code>file.seekg(100, ios::beg); // jumps to offset 100</code>
Seek Backward (End)	<code>file.seekp(-50, ios::end); // jumps 50 bytes before EOF</code>
Get Current Position	<code>int pos = file.tellg(); // returns read offset</code>

One-Line Definitions

- **File Stream:** The logical interface that connects the program to physical files on the disk.
- **ifstream:** A file stream class specialized in reading data from disk files.
- **ofstream:** A file stream class specialized in writing data to disk files.
- **fstream:** A stream class that supports both input and output file operations.
- **File Mode:** Settings that configure file permissions (read, write, binary, append) when a file is opened.
- **Get Pointer:** The read pointer that keeps track of the next index byte to read from a file.
- **Put Pointer:** The write pointer that keeps track of the next index byte to write to a file.
- **tellg():** A file function returning the current index offset of the read (get) pointer.
- **seekg():** A file function used to reposition the get pointer to a specified index location.
- **Binary Mode:** Opening a file to write raw memory bytes directly, bypassing text formatting.

Frequently Asked / Viva Questions

Q1. What is the difference between opening a file using constructor vs. the open() method?

The constructor automatically opens the file and is clean for single files. The

Frequently Asked / Viva Questions (contd.)

`open()` function allows opening multiple files sequentially using the same stream object.

Q2. Explain the difference between the `ios::app` and `ios::ate` file modes.

Both open the file with pointers positioned at the end. However, `ios::app` forces all write operations to occur at the end of the file, whereas `ios::ate` allows shifting the pointer to write anywhere in the file.

Q3. Why is `reinterpret_cast<char*>` mandatory in `read()` and `write()` functions?

These functions are designed to accept a stream of bytes (`char*`). We must cast our class object address to a byte pointer for binary transfer.

Q4. What is random file access and how is it implemented in C++?

Random access allows reading/writing at any arbitrary location in the file without processing it sequentially. It is implemented using `seekg()`, `seekp()`, `tellg()`, and `tellp()`.

Q5. What happens if we attempt to open a non-existent file for reading?

The file stream fails to open, and its error flag is set. We can verify this state using the `fail()` function or checking the stream's truth value: `if(!file)`.

Q6. Why must we close files explicitly before exiting the program?

Closing files flushes the stream buffers (saving unwritten data from RAM to disk) and releases system file handles.

Q7. Can we open a single file for simultaneous reading and writing?

Yes, using the `fstream` class and combining file modes: `fstream file("data.dat", ios::in | ios::out | ios::binary)`.

Q8. What is the purpose of the `eof()` function in C++ streams?

It returns `true` if the get pointer has reached the end-of-file (EOF) during a read operation.

Q9. How does text mode file access differ from binary mode file access?

Text mode translates line endings (like converting `\n` to `\r\n` on Windows) and parses values. Binary mode copies raw memory bytes bit-by-bit without any translations.

Q10. Which class acts as the root class for all file stream classes?

The root class is `ios` (specifically, `ios_base` is the grandparent class, and `ios` inherits from it).

Q11. How can we determine the total size of a file in bytes using seek functions?

We can seek to the end of the file and use `tellg()` to query the pointer offset: `file.seekg(0, ios::end); int size = file.tellg();`

Frequently Asked / Viva Questions (contd.)

Q12. What is a file stream buffer and why is it used?

It is a temporary block of memory in RAM that buffers read/write bytes to minimize slow disk access operations, improving performance.

Comparison: Sequential File Access vs. Random File Access

Feature	Sequential File Access	Random File Access
Pointer Mechanics	Pointers move step-by-step from beginning to end.	Pointers can jump directly to any arbitrary offset.
Speed	Slower for large databases (must read intermediate records).	Extremely fast (access time is constant, $O(1)$, using offsets).
File Functions	Uses standard operators like <code><<</code> , <code>>></code> , or <code>getline</code> .	Uses functions like <code>seekg()</code> , <code>seekp()</code> , <code>tellg()</code> , <code>tellp()</code> .
Common Modes	Text mode (comma/space-separated values).	Binary mode (fixed-size struct or class records).
Use Case	Reading configuration files or writing log lines.	Modifying specific records in a large binary database.

Module 9

Exception Handling Mechanism

1. Exception Handling Mechanism

Exception handling separates error detection from error resolution, allowing applications to recover gracefully from runtime errors.

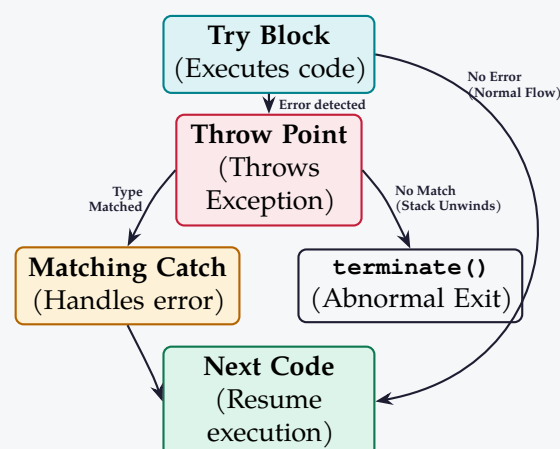
1.1 Basic Syntax: `try`, `throw`, and `catch`

- **try block:** Encloses the segment of code where exceptions might occur.
- **throw statement:** Triggers an exception when an anomaly is detected, exiting the try block immediately.
- **catch block:** Defines a handler that matches the data type of the thrown exception to resolve the error.

1.2 Exception Propagation & Stack Unwinding

If a throw point occurs inside a nested function, the compiler exits that function immediately. It searches up the function call stack for an active `try` block. During this process, the compiler invokes destructors for all local variables created inside the exited stack frames. This process is called **Stack Unwinding**.

Exception Handling Control Flow Map



Code Example: Exception throw point outside the main try block

```
#include <iostream>
// Function that throws exception
double divide(double a, double b) {
    if(b == 0.0) {
        throw "Division by zero condition!"; // Throws a const char*
    }
    return a / b;
}

int main() {
    double x = 10, y = 0, result;
    try {
        result = divide(x, y); // Throw point is inside divide()
        std::cout << "Result = " << result << std::endl;
    }
    catch(const char* msg) { // Catches the thrown char*
        std::cerr << "Caught Exception: " << msg << std::endl;
    }
    return 0;
}
```

1.3 Multiple Catches, Catch-All, and Re-throwing

- **Multiple Catches:** A single try block can have multiple sequential catch handlers to process different exception types.
- **Catch-All (`catch(...)`):** A special catch handler that intercepts any exception type. It must be placed at the very end of the handler list.
- **Exception Re-throwing:** An caught exception can be re-thrown using the `throw;` statement without arguments inside the catch block to pass it up to a higher scope.

2. Generic Programming using Templates

Templates allow writing code that is independent of data types, enabling the creation of generic functions and classes.

2.1 Function Templates

Function templates act as formulas for generating functions based on argument types.

Function Template Syntax

```
template <typename T>    Return_Type  FuncName(T args)  {...}    (28)
```

Code Example: Multi-parameter function template

```
#include <iostream>
// Multi-parameter function template
template <typename T1, typename T2>
void printComparison(T1 x, T2 y) {
    if(x > y) {
        std::cout << x << " is larger than " << y << std::endl;
    } else {
        std::cout << x << " is not larger than " << y << std::endl;
    }
}

int main() {
    printComparison(10, 5.5); // T1 = int, T2 = double
    printComparison(4.2, 9.8); // T1 = double, T2 = double
    return 0;
}
```

2.2 Class Templates

Class templates are used to create generic containers or data structures that operate on any data type.

Code Example: Generic Stack class template

```
#include <iostream>
template <typename T, int SIZE = 10> // typename T and non-type int SIZE
class Stack {
private:
    T arr[SIZE];
    int top;
public:
    Stack() : top(-1) {}

    bool push(T element) {
        if(top >= SIZE - 1) return false; // Overflow
        arr[++top] = element;
        return true;
    }

    T pop() {
        if(top < 0) throw "Stack Underflow!"; // Throw const char*
        return arr[top--];
    }
};

int main() {
    try {
        Stack<int, 5> intStack; // Instantiate an int stack of size 5
        intStack.push(10);
        intStack.push(20);
        std::cout << intStack.pop() << std::endl; // 20
        std::cout << intStack.pop() << std::endl; // 10
        std::cout << intStack.pop() << std::endl; // Throws exception
    }
    catch(const char* e) {
        std::cout << "Exception: " << e << std::endl;
    }
    return 0;
}
```

Quick Revision — Module IX

Exception Handling & Templates | OE-EC604C

Key Exception and Template Syntax

Try-Catch Block	<pre>try { throw 404; } catch(int err) { /* handles 404 */ }</pre>
Catch-All Handler	<pre>catch(...) { /* catches any exception type */ }</pre>
Re-throwing	<pre>catch(const char* e) { throw; // Re-throws exception up }</pre>
Function Template	<pre>template <typename T> T maxVal(T a, T b) { return (a > b) ? a : b; }</pre>
Class Template	<pre>template <class T> class Box { T value; };</pre>
Template Instantiation	<pre>Box<double> doubleBox; // Explicit template argument</pre>

One-Line Definitions

- **Exception:** A runtime error or anomaly that disrupts the normal execution flow of a program.
- **Stack Unwinding:** The automatic destruction of local variables in stack frames during exception propagation.
- **Catch-All Handler:** A fallback handler (`catch(...)`) that catches any exception type not previously intercepted.
- **Re-throwing:** Forwarding an caught exception up to the parent calling frame using the `throw;` statement.
- **Template:** A blueprint used by the compiler to generate type-specific functions or classes automatically.
- **Generic Programming:** A programming model focused on writing algorithms independent of specific data types.
- **Non-type Template Parameter:** A template parameter that represents a constant value (e.g., `int SIZE`) rather than a type.
- **Template Specialization:** Defining a unique template implementation for a specific data type (e.g., `T = char*`).
- **terminate():** A system function called when no matching catch handler is found for a thrown exception.

Frequently Asked / Viva Questions**Q1. What happens if an exception is thrown but no matching catch block is found?**

The runtime library calls the `terminate()` function, which by default calls `abort()` to terminate the program immediately.

Q2. Can we throw class objects as exceptions in C++?

Yes. Throwing class instances is the standard way to implement custom exceptions, allowing structured error data to be passed to handlers.

Q3. What is the purpose of the catch-all handler and where must it be placed?

`catch(...)` catches all exception types. It acts as a safety net and must be placed as the last handler, otherwise it blocks subsequent handlers.

Q4. Why does template instantiation not happen at class definition time?

A template only defines a blueprint. The compiler only generates class code when the template is instantiated with a specific type (e.g., `Stack<int>`).

Q5. Explain the compile-time overhead associated with templates.

Templates require the compiler to generate unique code for each type instantiation, which increases compile times and binary sizes (code bloat).

Q6. What is template function specialization?

It allows defining a custom implementation of a function template for a specific data type (e.g., optimizing string comparisons instead of pointer comparisons).

Q7. Can templates have default arguments?

Yes. Both class templates and function templates (in C++11) can have default type arguments (e.g., `template <typename T = int>`).

Q8. How does the compiler select between overloaded template functions and regular functions?

The compiler prefers a regular non-template function if it matches the arguments exactly. If no match is found, it instantiates the template.

Q9. Explain the difference between exception handling in C++ vs. Java.

C++ does not support checked exceptions (you do not have to declare exceptions in method signatures), and it does not have a native `finally` block (resource cleanup relies on RAII).

Q10. What is the difference between `typename` and `class` keywords in template declarations?

Inside template parameters, `typename` and `class` are completely equivalent and interchangeable. E.g., `template <class T>` and `template <typename T>` behave identically.

Q11. Why is it dangerous to throw pointers to local variables?

Local variables are destroyed when stack frames are unwound. The handler will receive a dangling pointer to invalid memory, causing undefined behavior.

Frequently Asked / Viva Questions (contd.)

Q12. What is the RAII (Resource Acquisition Is Initialization) pattern?

It is a design pattern where resources (files, memory, locks) are bound to the lifetime of local objects. Destructors automatically release resources during normal exit or stack unwinding, preventing leaks.

Comparison: Exception Handling vs. Traditional Error Return Codes

Feature	Exception Handling (try-catch)	Traditional Error Codes (if-else)
Separation	Clear separation between execution logic and error handling code.	Error checking logic is mixed directly into the main algorithm.
Propagation	Errors propagate automatically up the call stack until handled.	Errors must be explicitly returned and checked in every function frame.
Data Payload	Exceptions can pass complex objects containing detailed error info.	Limited to returning simple integer codes or status booleans.
Performance	Slower when exceptions occur (due to stack unwinding and V-Table lookup).	Minimal overhead (simple conditional branches).
Uncaught Errors	Leads to immediate program termination (safe fail-fast).	Can be silently ignored, causing latent bugs or data corruption.